



Memorial University of Newfoundland

Faculty of Engineering and Applied Science

ENGI-9839

Software Verification and Validation

Literature Summary

Student Name: MD Abdul Hadi Chowdhury

Student ID: 202583341

Course: ENGI-9839

Instructor: Raja Abbas

Submission Date: July 2026

1 John Hughes: Software Testing with QuickCheck

Hughes explains the principles behind QuickCheck, a property-based testing approach in which the tester specifies general properties rather than only individual test examples. QuickCheck repeatedly generates test data and checks whether the stated properties hold. The chapter also discusses the importance of designing suitable generators and shows how automatically generated test inputs can exercise a much larger input space than a small collection of manually selected examples. The use of shrinking is particularly important because it reduces a failing input to a smaller counterexample that is easier to understand and diagnose [1].

This source provided the conceptual foundation for the property-based component of the project. The Hypothesis test suite followed the same general model by defining financial properties, generating decimal amounts and account states, and checking those properties over many examples. The source was also useful for understanding an important limitation of property-based testing: automatic generation is effective only when the relevant behaviour has first been expressed as a property. In this project, the missing property for interest-calculation scheduling demonstrated that generated inputs cannot compensate for an incomplete property catalogue.

2 Yue Jia and Mark Harman: Development of Mutation Testing

Jia and Harman provide a broad survey of the development of mutation testing. They explain how mutation testing creates modified versions of a program by applying small syntactic changes, known as mutation operators. A test suite is considered effective when it detects these changes by causing the mutants to fail. The survey reviews mutation operators, equivalent-mutant challenges, execution costs, applications of mutation testing, and the relationship between mutation analysis and test adequacy [2].

This survey supported the methodological design of the project. It justified using mutation score as a stronger adequacy measure than structural coverage alone because mutation analysis evaluates not only whether code executes, but also whether the test oracle detects a behavioural change. It also informed the treatment of surviving mutants. Instead of interpreting every survivor as a test weakness, the project classified survivors as genuine

weaknesses, equivalent or likely equivalent mutants, underspecified diagnostic behaviour, or behaviour outside the selected scope.

3 Mike Papadakis et al.: Mutation Scores and Real Fault Detection

Papadakis and colleagues investigate whether mutation scores are correlated with the detection of real software faults. Their large-scale empirical study examines the extent to which mutation testing can serve as a proxy for actual fault-detection capability. The authors show that mutation scores can provide useful information about test effectiveness, but they also warn that the relationship can be influenced by confounding factors such as test-suite size and test selection [3].

This source was important for interpreting the project results cautiously. The manual suite achieved a higher controlled mutation score, but this does not prove that manually designed testing is universally more effective than property-based testing. The two suites differed in behavioural reach and in the number of test functions. Therefore, the report also calculated a reached-mutant score, which considered only killed and surviving mutants. The much smaller difference between the reached-mutant scores showed why mutation results should be interpreted together with coverage, execution reach, and test-design scope rather than as a single standalone percentage.

4 Savitha Ravi and Michael Coblenz: Property-Based Testing in Python

Ravi and Coblenz present an empirical evaluation of property-based testing in Python projects. Their work examines how property-based testing is used in practice and evaluates its effectiveness using mutation analysis. The study highlights both the potential benefits of generated testing and the difficulty of developing effective properties and input strategies. The results indicate that property-based testing can improve test effectiveness, but its success depends heavily on the quality, strength, and completeness of the properties defined by developers [4].

This research closely relates to the practical comparison conducted in the project. The Hy-

pothesis suite successfully evaluated important financial invariants across generated inputs, including deposit monotonicity, exact withdrawal subtraction, validation boundaries, and interest relations. However, mutation analysis showed that the property catalogue reached fewer behaviours than the manually designed suite. The source helped support the project’s conclusion that property-based testing is not automatically complete: it provides broad variation within represented behaviours, but omitted properties can leave entire methods or responsibilities untested.

5 saadm11 Banking System Repository

The `saadm11/banking-system` repository is an open-source banking application implemented using Python and Django. It includes functionality for account management, deposits, withdrawals, transaction records, interest calculation, reporting, and related banking operations. The repository was used as the subject system for the case study, and the evaluated source was pinned to commit `9c3ee3eea75030280405506c62d968d646981c05` to ensure that both test suites were evaluated against the same implementation [5].

The repository was suitable for this project because its financial operations contain observable validation rules, arithmetic state changes, persistence behaviour, and boundary conditions. These characteristics supported manual equivalence-partitioning and boundary-value tests as well as property-based financial invariants. The original repository contained no executable tests for the selected functionality, so the manual and property-based suites were created specifically for the experiment. The repository therefore served as the practical case study rather than as an academic source supporting general testing theory.

References

- [1] J. Hughes, “Software testing with quickcheck,” in *Central European Functional Programming School: Third Summer School, CEFP 2009, Revised Selected Lectures*, ser. Lecture Notes in Computer Science, Z. Horváth, R. Plasmeijer, and V. Zsók, Eds. Berlin, Heidelberg: Springer, 2010, vol. 6299, pp. 183–223.
- [2] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [3] M. Papadakis, S. Yoo, D. Shin, and D.-H. Bae, “Are mutation scores correlated with real fault detection? a large-scale empirical study on the relationship between mutants and real faults,” in *Proceedings of the 40th International Conference on Software Engineering*. Association for Computing Machinery, 2018, pp. 537–548.
- [4] S. Ravi and M. Coblenz, “An empirical evaluation of property-based testing in python,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3897–3923, Oct. 2025.
- [5] saadm11, “Banking system: A banking system concept created using the django python web framework and tailwind css,” <https://github.com/saadm11/banking-system>, gitHub repository; evaluated revision 9c3ee3eea75030280405506c62d968d646981c05; accessed July 2026.